

FINAL PROJECT REPORT

MindCare Oracle: An End-to-End AI Application with MLOps for Mental Health Risk Prediction

1. ABSTRACT

This project presents MindCare Oracle, an end-to-end artificial intelligence application developed to predict mental health treatment risk from structured survey responses. The central objective is not only to produce a predictive model, but to operationalize the complete AI product lifecycle using MLOps principles, including data processing automation, experiment tracking, model serving, observability, and reproducible deployment. The application is implemented as a multi-service architecture with a React frontend, FastAPI backend, MLflow experiment tracking, Airflow orchestration, DVC pipeline stages, and Prometheus-Grafana monitoring. This report documents the problem statement, system design, implementation choices, process flow, guideline compliance, and practical challenges encountered during final system stabilization.

2. INTRODUCTION

2.1 Background and Motivation

Mental health concerns are increasing across workplaces and academic environments. However, early indication of treatment need is frequently delayed due to social stigma, low awareness, and absence of quick screening support. In this context, data-driven risk estimation can provide a preliminary decision-support mechanism for timely intervention.

2.2 Problem Statement

The problem addressed in this project is:

"Given self-reported demographic and workplace-survey attributes, predict whether an individual is likely to require or seek mental health treatment."

The project therefore focuses on building a usable and technically robust predictive system for mental health danger/risk indication, where "danger" is operationally represented by model output indicating likelihood of treatment need.

2.3 Project Objectives

The major objectives were:

- To build a machine learning pipeline for reliable mental health risk prediction.
- To create a user-facing web interface usable by non-technical users.

- To implement a reproducible MLOps workflow covering data, model, and deployment lifecycles.
- To provide operational monitoring and drift-awareness mechanisms in a production-like setup.
- To package the complete system in a local, cloud-independent containerized environment.

3. SYSTEM OVERVIEW

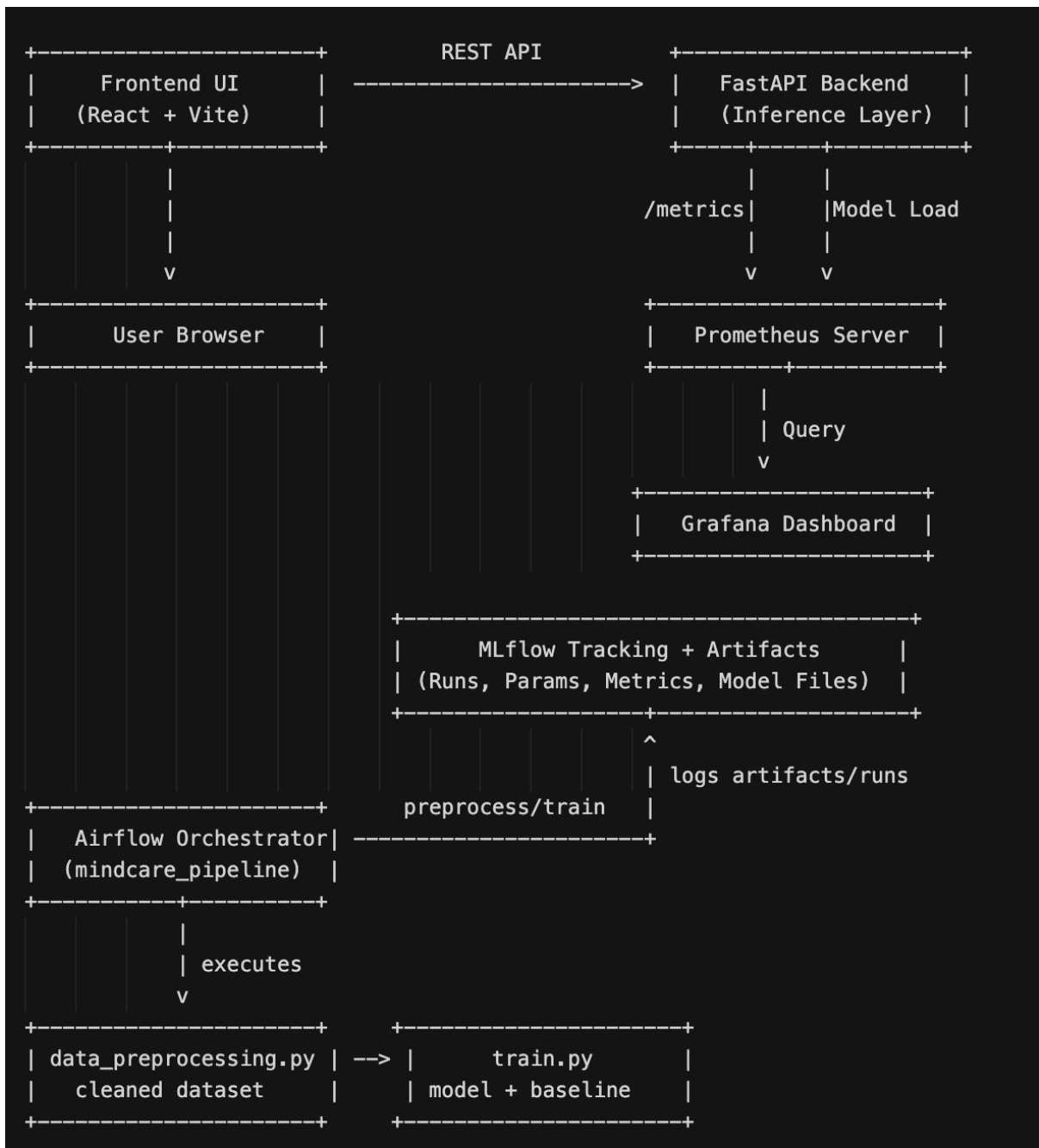
MindCare Oracle is organized as loosely coupled components connected through REST APIs and tool interfaces:

- Frontend Layer: React-based web application for user input and result display.
- Inference Layer: FastAPI service exposing prediction and operational endpoints.
- Training Layer: Python-based preprocessing and training scripts orchestrated by DVC and Airflow.
- Tracking Layer: MLflow for experiment metadata, parameters, metrics, and model artifacts.
- Monitoring Layer: Prometheus for metrics scraping and Grafana for visualization.
- Deployment Layer: Docker Compose to maintain environment parity and service orchestration.

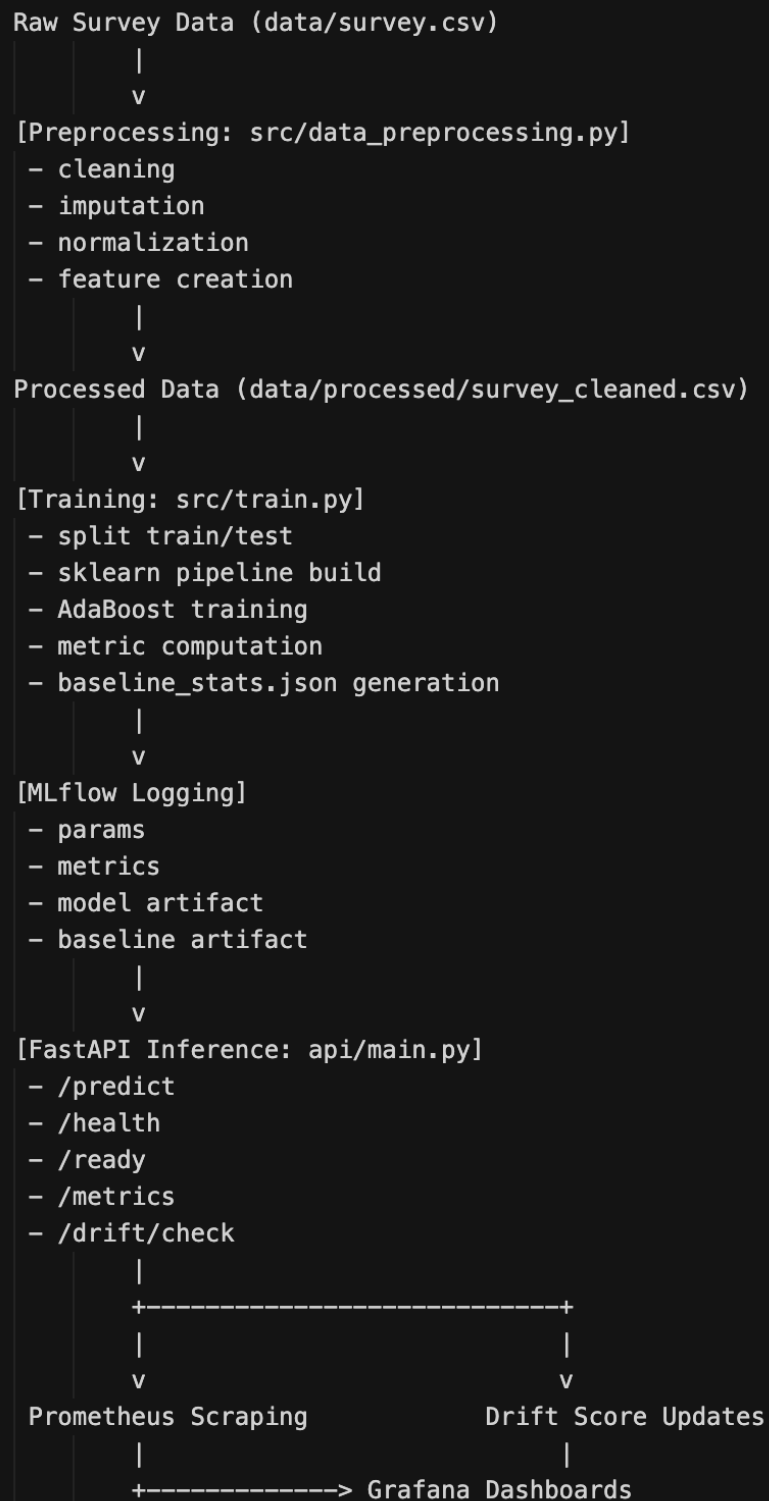
This architecture was selected to satisfy both software engineering principles (modularity, loose coupling) and MLOps requirements (automation, reproducibility, observability).

3.1 Architecture and Pipeline Diagrams

3.1.1 Overall System Architecture Diagram



3.1.2 Backend Pipeline Diagram (Data -> Training -> Serving -> Monitoring)



3.1.3 Frontend Interaction Pipeline Diagram

```

User opens web application (React frontend)
    |
    v
User enters features:
Age, Gender, family_history, benefits,
care_options, anonymity, leave, work_interfere
    |
    v
Frontend performs input validation and formatting
    |
    v
HTTP POST request to backend endpoint: /predict
    |
    v
FastAPI validates schema (Pydantic model)
    |
    v
Model inference returns:
- treatment (boolean)
- probability (float)
    |
    v
Frontend renders prediction outcome to user
    |
    v
In parallel: backend emits telemetry for metrics and monitoring

```

4. DETAILED PROCESS FLOW AND IMPLEMENTATION

4.1 Stage A: Problem Definition and Data Acquisition

The project starts from a supervised binary classification framing. The target variable is treatment need, derived from historical survey labels. The raw dataset is maintained under `data/survey.csv`.

Implementation process:

1. Define target and feature columns relevant to treatment risk.
2. Version source data and data-processing logic through repository structure and DVC stage definitions.
3. Establish measurable outputs in terms of model metrics (accuracy, ROC-AUC, precision, recall, F1) and system behavior (endpoint health, response availability).

Design choice rationale:

- A tabular survey-based dataset aligns naturally with interpretable classical ML methods.
- A binary classification target simplifies practical deployment for risk indication use cases.

4.2 Stage B: Data Preprocessing and Feature Engineering

Preprocessing is implemented in `src/data\_preprocessing.py` and includes:

- schema-level cleaning and column removal for irrelevant fields,
- missing value imputation for numerical and categorical features,
- categorical normalization (notably gender normalization across heterogeneous text variants),
- outlier handling for age values,
- feature enrichment through age-binning (`age\_range`).

Automation:

- DVC stage `process\_data` executes preprocessing reproducibly.
- Airflow task `preprocess\_data` includes the same operational step for orchestration.

Design choice rationale:

- Deterministic scripted preprocessing was chosen over manual notebook-only cleaning to ensure repeatability.
- Explicit normalization rules were preferred to reduce noise from free-text categorical entries.
- Feature engineering logic is separated from model-serving logic to maintain maintainability and auditability.

4.3 Stage C: Model Development and Training

Training is implemented in `src/train.py` using a compositional sklearn pipeline:

- `ColumnTransformer` for modality-specific transforms,
- `MinMaxScaler` for numerical normalization,
- `OrdinalEncoder` for categorical encoding with unknown-category handling,
- `AdaBoostClassifier` with shallow decision tree base learner.

Training and evaluation process:

1. Load processed dataset.
2. Split into train/test partitions.
3. Compute baseline feature statistics from training data for downstream drift reference.
4. Train model pipeline with parameterized estimator count.
5. Evaluate against test data.
6. Log run metadata and artifacts to MLflow.

Tracked MLflow entities:

- Parameters: model type, estimator count, random state, train/test sample counts.
- Metrics: accuracy, ROC-AUC, precision, recall, F1-score, confusion-matrix-derived measures.
- Artifacts: serialized model and baseline statistics file.

Design choice rationale:

- AdaBoost with shallow trees offers a balanced tradeoff between lightweight local execution and non-linear decision capacity.
- Unified sklearn pipeline reduces train-serving skew risk by preserving transformation consistency.
- MLflow integration ensures experiment traceability and comparability.

4.4 Stage D: Model Serving and Deployment

Serving is implemented in FastAPI (`api/main.py`) with endpoints:

- `POST /predict` for inference,
- `GET /health` for liveness,
- `GET /ready` for readiness verification based on model load status,
- `GET /metrics` for Prometheus scraping,
- `POST /drift/check` for input-distribution drift estimation.

Deployment process:

- API, frontend, airflow, prometheus, and grafana are containerized and orchestrated through `docker-compose.yml`.
- MLflow server is started and connected with local artifact storage.
- Startup sequence is managed through `start.sh` for rapid project bring-up.

Design choice rationale:

- FastAPI was selected for strong validation support, interactive API docs, and efficient REST serving.
- Health/readiness endpoints were included to align with orchestration best practices.
- Multi-container Compose deployment enforces service separation and environment parity.

4.5 Stage E: Monitoring, Drift, and Maintenance Workflow

Observability is implemented using Prometheus instrumentation and Grafana visualization. API metrics include request counts, latency, errors, prediction counters, and feature-level drift scores.

Drift process:

- Baseline statistics are generated during training and stored.
- Incoming inference records can be evaluated via `/drift/check`.
- Numeric drift and categorical distribution shift are computed through lightweight statistical scoring.

Maintenance process:

- Airflow DAG (`airflow/dags/mindcare_pipeline.py`) orchestrates preprocessing and retraining.
- Retraining can be triggered as a periodic or on-demand maintenance activity.

Design choice rationale:

- Drift signaling was included as an operational safeguard against silent model degradation.
- Airflow orchestration supports transition from ad-hoc retraining to managed lifecycle operations.

5. MLOPS GUIDELINE COMPLIANCE MAPPING

5.1 Core Principles

Automation:

Implemented through DVC stages, Airflow DAG tasks, scripted startup, and repeatable training pipeline execution.

Reproducibility:

Implemented through versioned code, codified pipeline stages, MLflow run tracking, and parameterized model training commands.

Continuous Integration Orientation:

Pipeline logic is CI-ready through deterministic scripts and DVC DAG; full hosted CI runner integration can be extended as next maturity step.

Monitoring and Logging:

Implemented using structured application logging and Prometheus-Grafana observability stack.

Collaboration:

Supported through maintained technical documentation (architecture, HLD, LLD, testing plan, user manual).

Version Control:

Implemented via Git-based source management, DVC stage definitions, and MLflow artifact lineage.

Environment Parity:

Implemented using Docker Compose with isolated service containers and reproducible local runtime.

5.2 Lifecycle Stage Compliance

Problem Definition and Data Collection:

Achieved through explicit business objective and structured dataset pipeline.

Data Preprocessing and Feature Engineering:

Achieved through scripted cleaning, transformation, and baseline generation.

Model Development and Training:

Achieved through parameterized training, measurable evaluation, and experiment logging.

Model Deployment:

Achieved through API-based serving, containerized deployment, and operational endpoint support.

Monitoring and Maintenance:

Achieved through exporter instrumentation, dashboard visibility, drift checks, and retraining orchestration foundations.

6. TESTING STRATEGY AND QUALITY ASSURANCE

Test assets are present across the major layers:

- ``tests/test_preprocessing.py`` for data cleaning and feature preparation logic.
- ``tests/test_model.py`` for model pipeline behavior and drift utility checks.
- ``api/test_api.py`` for endpoint behavior and validation scenarios.

The testing framework covers:

- schema and transformation consistency,
- model train/predict behavior,
- API contract validity and error handling behavior.

In the final stabilization window, the project environment used for rapid runtime recovery did not have ``pytest`` executable configured in-path. Consequently, the report records test assets and strategy comprehensively, while recommending a final environment-prepared execution snapshot before viva submission.

7. CHALLENGES FACED AND RESOLUTION

Challenge 1: Runtime startup failure due to Docker access permissions

- Observation: service startup initially failed because Docker daemon socket access was denied in the restricted execution context.
- Resolution: startup was re-executed in an environment with proper Docker permissions, after which all core services came up successfully.

Challenge 2: Port communication ambiguity

- Observation: frontend service mapping and startup message could cause confusion regarding which host port to use during demonstration.
- Resolution: operational verification was performed against active mapped ports, and demonstration guidance should clearly mention the effective frontend endpoint.

Challenge 3: Production-hardening scope limitations

- Observation: local academic deployment prioritizes operability and MLOps lifecycle demonstration; enterprise-grade encryption and fully automated rollback/feedback loops remain extension areas.
- Resolution: these limitations are documented transparently as future work, with architecture already prepared for incremental enhancement.

8. KEY DESIGN DECISIONS AND JUSTIFICATION

Choice 1: Classical ML over heavy deep models

Reason: tabular structured inputs and local resource constraints favor robust, lightweight models with faster iteration and reproducibility.

Choice 2: FastAPI-based model serving

Reason: strong validation support, clean API contracts, and straightforward integration with monitoring.

Choice 3: DVC + Airflow + MLflow integration

Reason: this combination provides practical coverage across data pipeline management, orchestration, and experiment traceability without cloud dependency.

Choice 4: Docker Compose microservice separation

Reason: enforces loose coupling and environment parity while enabling independent subsystem troubleshooting.

Choice 5: Drift baseline persistence

Reason: model monitoring should include input-distribution monitoring, not only endpoint uptime metrics.

9. CURRENT SYSTEM STATUS (SUBMISSION READINESS)

At the end of stabilization, the following status was achieved:

- Airflow service: running
- API service: running and health endpoint responsive
- Frontend service: running and reachable
- Prometheus service: running
- Grafana service: running
- MLflow server: started for experiment visibility

This confirms that the project is demonstration-ready as an integrated MLOps application.

10. LIMITATIONS AND FUTURE ENHANCEMENTS

Current limitations:

- final automated test execution report depends on environment-level test runner readiness,
- enterprise-grade encryption at rest/in transit can be strengthened,
- full closed-loop real-world label feedback automation can be expanded,
- rollback automation can be formalized beyond manual artifact selection.

Recommended future work:

- add hosted CI pipeline for automated test and lint gates,
- implement persistent feedback-label capture and retraining triggers,
- extend explainability layer (feature attribution summaries) for decision transparency.

11. CONCLUSION

MindCare Oracle successfully demonstrates a complete AI application lifecycle implemented with practical MLOps discipline. The project moves beyond isolated model training and delivers an operational system with reproducible data/model pipelines, API serving, monitoring instrumentation, orchestration capability, and containerized deployment. The mental health danger prediction objective is addressed through a robust and explainable workflow suitable for academic evaluation and extensible toward production-grade maturity.